

Portfolio and Applied Labs — Complete Project Manual

Project: Deepan Karthick's portfolio upgrade

Manual snapshot: 2026-08-22

Scope: two flagship applications, one 30-project Applied Labs application, the portfolio shell, verification, deployment, and maintenance

Source-of-truth boundary: this manual is based on the repository's project-control status, decisions, specifications, lab records, batch reports, canonical manifest, source tree, and package/build scripts. It does not invent production metrics, user counts, performance results, or deployment claims.

Final release:

<https://deepan-karthick-portfolio.industrious-keyboard.workers.dev/> on Cloudflare Workers Static Assets. The homepage, Applied Labs, Timber demo, and manual PDF returned HTTP 200 after deployment; exact deployment IDs are recorded in the release handoff rather than embedded in this versioned artifact.

1. What this project is

This mission turns a list of 30 completed credentials into evidence of applied work. The portfolio homepage is intended to feature only two substantial flagship applications:

- 1 **Printing Press ERP** — a public, synthetic, browser-local demonstration of a printing-business workflow, with a separately protected live mode for the private system.
- 2 **Timber CFT Pro with Billing** — a sanitized timber measurement and invoice application that preserves the supplied business rules without publishing client data.

The remaining 30 certificate-linked projects live in one **Applied Labs** application. Each lab has its own stable route but shares one catalog, one visual shell, and one deployment surface. The labs are not certificate thumbnails or generic quizzes. Each is a small working tool for a named or clearly identifiable user, with realistic inputs, deterministic decision logic, a useful output, validation, sample/reset controls, tests, and an honest limitation statement.

What “functional first” means

The project deliberately separates correctness from final visual polish. The implementation order is:

- 1 lock scope, privacy, and business rules;
- 2 create a realistic input and useful output;
- 3 make the workflow deterministic and inspectable;
- 4 add validation, example data, reset, and copy/download/print handoff where relevant;
- 5 test the pure domain logic;
- 6 integrate a lazy route and verify the rendered workflow;
- 7 only after all functional batches, redesign the portfolio and labs visually;
- 8 then update resumes, deploy, run production regression checks, and finish user documentation.

This order reduces the chance that visual work hides incomplete behavior. It also allows each batch to be tested and repaired before more projects are added.

Current verified state at this snapshot

Area	Recorded state	Important boundary
Printing Press ERP	Functional public demo; claimed live and re-verified on Cloudflare	Source is in a separate private repository, not this repository; dependency advisories remain deferred
Timber CFT Pro with Billing	Functional sanitized static demo; tests, build, and realistic browser workflow passed	Production hosting for this rebuilt demo was deferred in the Batch 1A record; the final Cloudflare production mapping still needs its release smoke test
Labs 01–24 and 26	Functional; automated, build, browser, responsive, and console evidence recorded by completed batches	The redesigned catalog source is complete; its final production-build and browser regression are pending
Labs 25 and 27–30	Implemented, documented, lazy-route integrated, manifest-marked functional; cumulative 151-test checkpoint passed	Final central production-build and browser/mobile/console acceptance remains pending in STATUS.md
Portfolio homepage	Two-flagship/30-lab editorial redesign is implemented in source	Final production-build, responsive browser, link, and console acceptance remain pending

The repository root README.md still describes an earlier research prototype. For this mission, docs/project-control/, docs/project-control/project-manifest.json, and this manual are the more current records.

2. Delivery strategy for a large project

The work is split into bounded batches so failures stay local and the computer is not asked to build everything concurrently.

Batch	Scope	Recorded checkpoint
0	repository control plane, manifest, Applied Labs shell	architecture and validation foundation
1A	Timber CFT Pro with Billing	9 Timber tests; combined build and browser flow passed
1B	Printing Press ERP repair and Cloudflare deployment	6 runtime/store tests, lint, build, browser routes, persistence, and clean console passed
2	Labs 01–05	30 cumulative Node tests; builds and browser acceptance passed
3	Labs 06–10	53 cumulative tests; builds and browser acceptance passed
4	Labs 11–14 and 26; Lab 03 usefulness repair	78 cumulative tests; builds and browser acceptance passed
5	Labs 15–19	104 cumulative tests; builds and browser acceptance passed
6	Labs 20–24	126 cumulative tests; builds and browser acceptance passed
7	Labs 25 and 27–30	151 cumulative tests passed; final central build/browser checkpoint still pending
8	portfolio and labs visual redesign	source implementation complete; final browser gate pending
9	ATS and creative/startup resume refresh	one-page DOCX/PDF outputs generated and verified with the final portfolio URL
10	production deployment, regression, health checks, final manual	unified Cloudflare package deployed; four public release URLs returned HTTP 200

Parallel agents may be used only on isolated, non-overlapping files. Shared routing, manifests, styles, status, and deployment are coordinator-owned integration points. The practical rule is: one implementation batch at a time, a focused test after each lab, then one central test/build/browser gate.

3. Architecture and repository map

project/	
├─ portfolio-src/	Portfolio source (Vite root)
│ ├─ index.html	Main portfolio entry
│ ├─ links/index.html	Links entry
│ ├─ dashboard/index.html	Dashboard entry
│ ├─ main.jsx	Main React portfolio source
│ └─ public/	Static portfolio assets/data
├─ labs-src/	Applied Labs source (separate Vite app)
│ ├─ main.jsx	Catalog, filters, lazy routes, lab registry
│ ├─ styles.css	Shared Applied Labs presentation
│ └─ labs/	Pure domain modules + React lab components
├─ timber-src/	Sanitized Timber public demo source
│ ├─ App.jsx	Invoice workflow
│ ├─ domain/calculations.js	Locked measurement rules
│ ├─ domain/invoice.js	Billing and validation rules
│ └─ storage.js	Browser-local persistence
├─ tests/	Node domain tests for Timber and Labs 01-30
├─ scripts/	
│ └─ validate_project_manifest.mjs	
├─ docs/project-control/	Decisions, specs, status, lab and batch records
├─ portfolio/	Generated portfolio build output
├─ labs/	Generated Applied Labs build output
├─ timber-demo/	Generated Timber build output
├─ vite.config.js	Portfolio multi-page build
├─ vite.labs.config.js	Applied Labs build
├─ vite.timber.config.js	Timber demo build
└─ package.json	Local scripts and dependencies

The Printing Press ERP source is recorded as living in a separate private `lakshmipriya-erp` repository. This repository contains the specification, verification report, portfolio screenshot, and deployed URL, but not the private ERP source tree. That separation should be preserved.

Runtime design

- **Portfolio:** React 19 + Vite 6, with Framer Motion and GSAP in the existing source.
- **Applied Labs:** React/Vite single-page application using hash routes such as `#/lab/mini-rag-studio`. Each lab component is imported lazily, so one lab does not force all lab components into the initial route chunk.
- **Timber demo:** separate React/Vite static application. It has no required server and stores synthetic invoices in the current browser.
- **Printing ERP demo:** standalone application deployed with Cloudflare Workers Static Assets. Its public demo runtime is browser-local and must create no Supabase client or request.
- **Domain logic:** labs generally separate pure `.js` functions from `.jsx` presentation, allowing Node tests without a browser.
- **Network boundary:** the core lab calculations are local and do not require API keys, model calls, cloud accounts, wallets, scanners, or paid services.

4. Local setup, run, test, and build

Prerequisites

- Node.js and pnpm must be installed.
- The exact supported Node.js version is **not pinned in the current `package.json`**, so use a currently maintained Node.js release compatible with Vite 6 and confirm with the full test/build commands.
- No environment variable is required for the portfolio, Applied Labs, or public Timber demo.

- Do not place private SQLite databases, real customer exports, credentials, or ERP live-mode secrets in this repository.

Install

```
pnpm install
```

The lockfile is `pnpm-lock.yaml`. Prefer a lockfile-respecting install in automated environments. If a clean CI command is adopted later, document its exact `pnpm` behavior rather than assuming it here.

Development servers

Run one surface at a time unless there is a specific integration reason to run more:

```
pnpm run dev:portfolio
pnpm run dev:labs
pnpm run dev:timber
```

`pnpm run dev` is an alias for `dev:portfolio`. Each command binds to `127.0.0.1`; use the port Vite prints in the terminal.

Tests

```
pnpm run test
```

This first validates that the canonical manifest contains the expected 30 labs and 2 flagships, then runs all `tests/*.test.mjs` files with Node's built-in test runner. An older Python research-prototype test is outside this public-surface command.

Run a focused lab test while developing:

```
node --test tests/lab24-feature-misuse-contract.test.mjs
```

Replace the filename with the lab being changed. Timber's focused domain suite is:

```
node --test tests/timber-domain.test.mjs
```

Production builds

```
pnpm run build
```

The combined command runs tests first and then creates all three public build outputs:

- `pnpm run build:portfolio` → `portfolio/`
- `pnpm run build:labs` → `labs/`
- `pnpm run build:timber` → `timber-demo/`

Preview the built surfaces with:

```
pnpm run preview
pnpm run preview:labs
pnpm run preview:timber
```

The Printing ERP has its own private repository, commands, and deployment configuration. Those commands are not present in this repository and should not be guessed.

5. Flagship 1 — Printing Press ERP

Problem and users

The flagship demonstrates a printing business that needs owner visibility and coordinated handling of orders, production, staff, inventory, invoices, designs, reverse requests, notifications, and reports. The public audience is a portfolio reviewer evaluating the product workflow. The private audience is the real business team using authenticated roles and a Supabase backend.

Why two runtime modes exist

The application has an explicit security boundary:

- demo: public, synthetic, browser-local data, no Supabase client or request, disposable reviewer edits.
- live: private, authenticated Supabase data, real role permissions, fail-closed behavior when configuration/authentication/backend access fails.

The application must never silently fall back from failed live mode to synthetic demo data; that could hide a real operational failure. It must also never let public demo authentication reach the live backend.

Main public workflow

- 1 Open the owner dashboard and see populated synthetic metrics without an endless loading state.
- 2 Open the production board and see fictional jobs.
- 3 Create a synthetic client order.
- 4 Reopen the order.
- 5 change its priority or production stage.
- 6 Navigate or reload and confirm the browser-local change persists.
- 7 Reset the public demo to its synthetic seed.

Secondary routes for staff, inventory, notifications, designs, invoices, reverse requests, and reports must at least load deterministic fictional data in demo mode.

Business and runtime rules

- Public demo records are clearly fictional and stored only in the current browser.
- Public demo mode creates zero Supabase clients or calls, even if hosting contains live environment variables.
- Invalid/corrupt local demo state recovers safely without touching live data.
- Private live mode requires valid configuration and authentication.
- Live routes enforce real role access and show bounded, recoverable failures rather than endless skeletons.
- No owner bypass is allowed.

Privacy and data boundaries

The public demo must not contain real customer names, phone numbers, GST numbers, orders, invoices, or uploaded designs. Reviewer-created demo records stay in the reviewer's browser and are disposable. Live customer/business data remains in the separately controlled private system.

Verification evidence

The Batch 1B report records:

- 6 runtime/local-store tests passed;
- ESLint passed;
- production build passed with 2,451 modules;
- /owner loaded five synthetic orders;
- production priority changes persisted after reload;
- inventory, invoices, reports, and staff routes loaded;
- direct-route SPA fallback worked on Cloudflare;
- no warnings or errors appeared in the accepted browser flow.

UI create-order submission was not automated because the native date input was incompatible with that browser-automation path. Creation and persistence are covered by unit tests, but this is a known automation gap rather than a hidden pass.

Deployment

- Platform: Cloudflare Workers Static Assets
- Recorded public route: <https://lakshmipriya-erp-demo.industrious-keyboard.workers.dev/owner>
- Recorded Cloudflare version: 4bd9ebd0-69d8-4c3f-b238-947fde815cc5
- Recorded repository commit: 6048565 (chore: add Cloudflare worker deployment)
- Status in project control: claimed live and re-verified after ownership handoff

Known limitations and maintenance work

- The public demo is not a multi-user server system and does not prove the private backend is healthy.
- Browser-local changes do not synchronize across devices.
- The dependency audit reported 2 moderate and 5 high advisories; remediation was deferred to a separate tested maintenance pass.
- Recurring production health checks were deferred to the final deployment batch.
- The actual ERP source and commands are outside this repository; maintain its own private runbook.

6. Flagship 2 — Timber CFT Pro with Billing

Problem and users

The application serves a timber business that measures pieces using its established “business inch” convention and needs those measurements converted into invoice quantities and financial totals. The public portfolio user can operate a complete synthetic invoice without receiving or exposing the client's Windows/SQLite data.

Product editions

- **Private Windows edition:** supplied Express/SQLite application for offline client use.
- **Public portfolio demo:** sanitized static React application with synthetic sample data and browser-local storage only.

The private SQLite database and bundled runtime must never be committed or deployed. Repository ignore rules cover .db, .db-wal, and .db-shm.

Locked measurement rules

These are business rules, not ordinary physical conversions, and must not be “corrected” without client approval and golden-test changes:

- 1 Business inches = millimetres ÷ 20, not ÷ 25.4.
- 2 A positive dimension rounds upward to the next 0.5 inch.
- 3 The minimum positive billing dimension is 1.5 inches.
- 4 $\text{CFT} = \text{rounded thickness inches} \times \text{rounded width inches} \times \text{length feet} \times \text{pieces} \div 144.$
- 5 $\text{ICBM} = \text{CFT} \div 35.315.$
- 6 M3 uses raw millimetres and actual length in metres.
- 7 PF length may be entered in feet or millimetres.

The calculation rules live in `timber-src/domain/calculations.js` and are protected by `tests/timber-domain.test.mjs`.

Billing rules

The invoice accepts bill number, party/customer, date, lorry number, item type, CFT or M3 pricing basis, rate, discount, GST percentage, and paid amount.

```

row amount      = selected measurement quantity × rate
subtotal        = sum of rounded row amounts
taxable amount  = max(0, subtotal - discount)
GST             = taxable amount × GST percentage
grand total     = taxable amount + GST
balance         = max(0, grand total - paid)
change due      = max(0, paid - grand total)

```

Rate, discount, and paid amount must be non-negative. GST is bounded from 0 to 100. Measurement values and piece counts have strict safe-demo bounds; zero, negative, non-integer piece counts, and unsafe ranges are rejected.

User workflow

- 1 Load the synthetic example or start a blank invoice.
- 2 Enter bill and customer metadata using synthetic values in the public demo.
- 3 Add, edit, or remove measurement rows.
- 4 Observe CFT, M3, row amount, and invoice totals update.
- 5 Correct validation errors before saving.
- 6 Save to browser-local history.
- 7 Search and reopen the invoice.
- 8 Delete only after confirmation.
- 9 Print or use the browser's Save as PDF action.

Privacy and storage

The public demo has no account, server database, analytics requirement, or cloud synchronization. Its invoices stay in the current browser. Corrupt local state recovers safely. The interface must tell users to use synthetic data.

Verification evidence

The Batch 1A report records 9 Timber tests, manifest validity, and a passing combined production build. The realistic browser sample produced:

- 12 pieces;
- 15.834 CFT;

- 0.240 M3;
- INR 34,270.96 grand total.

Browser-local save, reset, and reopen restored the same invoice and totals. These are verification fixtures, not production usage metrics.

Deployment and limitations

The rebuilt static output is `timber-demo/`. Batch 1A explicitly deferred production hosting and recurring health checks to Batch 10. The current portfolio source links to `https://calculator00.pages.dev/`, but the available project-control records do not prove that this URL is the exact sanitized Batch 1A build. Treat that association as **unverified until a production smoke test and version record are added**.

Other limitations:

- browser-local records are not shared or backed up;
- print layout depends on the browser's print engine;
- business formula changes require new approved golden vectors;
- the public demo is not authorization to migrate real client records;
- final visual polish remains deferred.

7. Applied Labs platform and UX

The Applied Labs application is a catalog rather than 30 separately maintained websites. Its main source is `labs-src/main.jsx`.

Catalog workflow

- Browse all 30 manifest-backed project cards.
- Search by title, credential, or relevant catalog text.
- Filter by AI/ML, cloud, programming/data, business/finance, or cybersecurity.
- Open a stable hash route: `#/lab/<slug>`.
- Return to the catalog or main portfolio.
- While a lab chunk loads, React Suspense provides a loading state.

The catalog and routes are derived from the canonical manifest/registry, while every lab's domain logic remains in its own module. The first 24 labs and Lab 26 have completed browser evidence in batch reports. The catalog itself was checked at desktop and 390×844 , and the cybersecurity filter was recorded returning seven labs. Batch 7's final rendered checks are still pending.

Shared UX contract for every lab

Each lab should provide:

- a clear real user and problem;
- bounded, labelled inputs with useful guidance;
- a realistic example and a reset action;
- validation that withholds unsafe or meaningless output;
- visible decision rules or calculations;
- a useful artifact such as a plan, memo, contract, code starter, test set, export, or comparison;
- privacy and safety boundaries near the workflow;

- keyboard-usable controls and a usable narrow-screen layout;
- a focused Node test for the domain module;
- an honest statement of what the lab cannot prove.

Route convention

Local and deployed routes use the hash form:

```
<Applied Labs base URL>/#/lab/<slug>
```

The base production URL is **not recorded as deployed in the current project-control status**, so this manual does not invent one.

8. Labs 01–10 — AI and machine learning

Lab 01 — AI Workflow Canvas

- **Certificate connection:** Generative AI for Everyone.
- **Real user/problem:** a learner or small project planner has an AI idea but needs to turn it into an ordered, reviewable workflow rather than an unsupported “add AI” statement.
- **Why built:** to demonstrate that useful AI planning begins with data, task, review, and risk boundaries.
- **Inputs:** goal, data source, task, and human-review level.
- **Decision logic:** validates required context, selects and orders seven workflow steps, adjusts review controls, and identifies overreach/data/quality risks.
- **Outputs/artifacts:** ordered workflow, risk register, and readiness checklist.
- **Safety/privacy:** local deterministic planning; no model, upload, API, production data inspection, or readiness certification.
- **Tests:** `tests/lab01-ai-workflow-canvas.test.mjs` covers the realistic sample, required fields, step ordering, review behavior, and overreach controls.
- **Limitations:** generated steps are a planning starter; they do not prove data quality, model suitability, permission, cost, or production readiness.
- **Route/files:** `#/lab/ai-workflow-canvas`; `aiWorkflowCanvas.js`; `AIWorkflowCanvas.jsx`.

Lab 02 — Mini RAG Studio

- **Certificate connection:** Generative AI with Large Language Models.
- **Real user/problem:** a learner or analyst wants to understand how retrieval supplies evidence for an answer without sending a document to a hosted model.
- **Why built:** to make chunking, retrieval, ranking, extraction, and citations inspectable.
- **Inputs:** a bounded pasted corpus and a question.
- **Decision logic:** makes overlapping chunks, tokenizes text, ranks chunks by transparent lexical overlap with deterministic tie-breaking, and extracts evidence-bearing sentences.
- **Outputs/artifacts:** extractive answer, ranked evidence, relevance scores, and chunk citations.
- **Safety/privacy:** runs locally; no upload, embedding API, LLM, account, or network request.
- **Tests:** `tests/lab02-mini-rag-studio.test.mjs` covers overlap, ranking, ties, citations, empty/invalid input, and deterministic output.
- **Limitations:** lexical retrieval misses semantic matches; extraction cannot synthesize new facts and is not a production RAG evaluation.

- **Route/files:** `#!/lab/mini-rag-studio; miniRagStudio.js; MiniRagStudio.jsx`.

Lab 03 — GAN Latent Gallery

- **Certificate connection:** Build Basic Generative Adversarial Networks (GANs).
- **Real user/problem:** a designer or student needs a reusable abstract background asset and wants to explore nearby latent-style variations reproducibly.
- **Why built:** to provide a useful visual export while honestly demonstrating latent-coordinate ideas without pretending a trained GAN is present.
- **Inputs:** bounded X/Y latent coordinates, seed, style, and selected main/nearby variation.
- **Decision logic:** deterministically maps coordinates, seed, and style into an SVG composition and four nearby variations; selection fixes the artifact signature.
- **Outputs/artifacts:** preview gallery and downloadable 1600×1600 SVG with accessible title plus seed/style/coordinate/signature metadata and a useful filename.
- **Safety/privacy:** browser-local generation; no model download, training data, API, upload, or trained-GAN claim.
- **Tests:** `tests/lab03-gan-latent.test.mjs` covers determinism, coordinate sensitivity, bounds, selection/export stability, XML escaping, metadata, and filename generation.
- **Limitations:** this is a deterministic latent-space simulation, not adversarial training or learned image generation.
- **Route/files:** `#!/lab/gan-latent-gallery; ganLatentModel.js; GanLatentGallery.jsx`.

Lab 04 — CNN Feature Explorer

- **Certificate connection:** Convolutional Neural Networks.
- **Real user/problem:** a learner needs to see exactly how a small image kernel turns pixels into a feature map.
- **Why built:** to replace black-box CNN language with inspectable multiplication, summation, and normalization.
- **Inputs:** editable 5×5 grayscale values or preset, plus edge, sharpen, or blur kernel.
- **Decision logic:** validates pixel/kernel ranges and performs one stride-1, no-padding 3×3 valid convolution, then normalizes the output for display.
- **Outputs/artifacts:** numeric 3×3 feature map and visual intensity map.
- **Safety/privacy:** entirely local; no image upload, trained model, classifier, inference claim, or API.
- **Tests:** `tests/lab04-cnn-convolution.test.mjs` covers edge and blur results, validation, output ranges, normalization, and flat-map handling.
- **Limitations:** one educational convolution pass cannot represent training, pooling, learned filters, multiple channels, or classification.
- **Route/files:** `#!/lab/cnn-feature-explorer; cnnConvolution.js; CnnFeatureExplorer.jsx`.

Lab 05 — Attention Text Explorer

- **Certificate connection:** Natural Language Processing with Attention Models.
- **Real user/problem:** a learner wants to see why one token receives more attention than another without opaque learned weights.
- **Why built:** to expose a scaled-dot-product-style calculation, positional influence, and softmax normalization step by step.
- **Inputs:** a bounded sequence of up to 12 tokens and a selected or entered focus token.
- **Decision logic:** derives transparent token features, calculates focus similarity plus positional bias, uses numerically stable softmax, and ranks normalized weights deterministically.
- **Outputs/artifacts:** score table, ranked weights, heat-style token sequence, and plain-language summary.

- **Safety/privacy:** local only; explicitly an attention visualization, not a transformer or language model.
- **Tests:** `tests/attention-text-explorer.test.mjs` covers tokenization, limits, repeatability, softmax stability/normalization, ranking, and a focus token outside the sequence.
- **Limitations:** transparent handcrafted features and position bias do not reproduce learned embeddings, heads, layers, or model behavior.
- **Route/files:** `#!/lab/attention-text-explorer`; `attention-text-explorer.js`; `AttentionTextExplorer.jsx`.

Lab 06 — Neural Network Playground

- **Certificate connection:** Neural Networks and Deep Learning.
- **Real user/problem:** a learner needs to trace how inputs, weights, and bias produce a neuron's output.
- **Why built:** to turn the neuron equation into visible arithmetic and small binary predictions.
- **Inputs:** two inputs, two weights, bias, and step, sigmoid, or ReLU activation.
- **Decision logic:** calculates each weighted contribution, sums the bias, applies the chosen activation, and repeats the configured neuron over four binary input pairs.
- **Outputs/artifacts:** equation, contribution breakdown, pre-activation value, activation result, and four sample predictions.
- **Safety/privacy:** deterministic and local; no training, model API, dataset, or performance claim.
- **Tests:** `tests/lab06-neural-network-playground.test.mjs` covers activations, weighted arithmetic, an OR-style configuration, and invalid numbers.
- **Limitations:** one manually configured neuron is not a multilayer network and does not learn.
- **Route/files:** `#!/lab/neural-network-playground`; `neuralNetworkPlayground.js`; `NeuralNetworkPlayground.jsx`.

Lab 07 — GenAI Lifecycle Explorer

- **Certificate connection:** Introduction to Generative AI.
- **Real user/problem:** a project planner needs to see what evidence and controls are required between an idea and a monitored release.
- **Why built:** to challenge “prototype equals production” thinking with explicit gates and exit artifacts.
- **Inputs:** use case, data readiness, impact/risk choices, and deployment context.
- **Decision logic:** applies a transparent 0–100 readiness heuristic, detects blockers, and orders six delivery stages with required artifacts and governance actions.
- **Outputs/artifacts:** readiness band, blockers, six-stage lifecycle, exit evidence, and governance requirements.
- **Safety/privacy:** planning only; no dataset inspection, model call, provider approval, legal conclusion, or deployment decision.
- **Tests:** `tests/lab07-genai-lifecycle-explorer.test.mjs` covers stage order, high-impact controls, missing-data blockers, scoring, and validation.
- **Limitations:** the score is an educational heuristic and must not be used as production authorization.
- **Route/files:** `#!/lab/genai-lifecycle-explorer`; `genAilifecycleExplorer.js`; `GenAilifecycleExplorer.jsx`.

Lab 08 — AI Opportunity Scorer

- **Certificate connection:** AI for Everyone.

- **Real user/problem:** a business planner needs an explainable first-pass screen for whether a repetitive task is a suitable AI candidate.
- **Why built:** to keep data readiness, error cost, privacy, and human judgment beside automation potential.
- **Inputs:** bounded 1–5 ratings for frequency, repetition, data readiness, error impact, privacy sensitivity, and judgment intensity.
- **Decision logic:** applies a visible weighted formula to a 100-point scale, maps the result to a recommendation band, and adds risk-specific guardrails.
- **Outputs/artifacts:** score, band, contribution breakdown, rationale, and next-step guardrails.
- **Safety/privacy:** local; no business data upload, ROI forecast, investment recommendation, or implementation approval.
- **Tests:** `tests/lab08-ai-opportunity-scorer.test.mjs` covers the example, inverse scenarios, high-risk guardrails, bounds, and deterministic scoring.
- **Limitations:** supplied ratings are subjective and the score cannot prove value, feasibility, permission, cost, or safety.
- **Route/files:** `#/lab/ai-opportunity-scorer`; `aiOpportunityScorer.js`; `AiOpportunityScorer.jsx`.

Lab 09 — Prompt Workbench

- **Certificate connection:** Prompt Engineering for ChatGPT.
- **Real user/problem:** a prompt author needs a structured, reusable prompt and a visible completeness check before using it elsewhere.
- **Why built:** to demonstrate role/task/context/constraints/example/format composition without pretending to judge a model response.
- **Inputs:** role, task, context, constraints, example, and output format.
- **Decision logic:** assembles the sections, applies six deterministic quality checks, calculates a local structure score/band, and suggests missing details.
- **Outputs/artifacts:** copy-ready prompt, check results, score/band, and improvement suggestions.
- **Safety/privacy:** no model call; entered prompt text remains local and should not contain secrets.
- **Tests:** `tests/lab09-prompt-workbench.test.mjs` covers deterministic assembly, strong and incomplete examples, improvements, empty input, and validation.
- **Limitations:** structural checks do not measure correctness, safety, or response quality from any specific model.
- **Route/files:** `#/lab/prompt-workbench`; `promptWorkbench.js`; `PromptWorkbench.jsx`.

Lab 10 — ML Model Lab

- **Certificate connection:** Machine Learning Specialization.
- **Real user/problem:** a student or analyst needs to understand a complete small classification decision, including scaling, neighbours, votes, and evaluation.
- **Why built:** to make K-nearest-neighbours inspectable rather than returning only a label.
- **Inputs:** bounded CSV-style rows with two numeric features and a label, k, and query X/Y values.
- **Decision logic:** parses and validates rows, standardizes both features, computes Euclidean distances, selects neighbours, uses deterministic vote/tie rules, and performs leave-one-out evaluation.
- **Outputs/artifacts:** predicted label, nearest-neighbour table, vote evidence, standardization details, leave-one-out accuracy, and confusion summary.
- **Safety/privacy:** local data only; no upload, training service, API, or production-quality claim.
- **Tests:** `tests/lab10-ml-model-lab.test.mjs` covers parsing/errors, scaling, neighbours, deterministic ties, evaluation, and invalid k.

- **Limitations:** exactly two numeric features, bounded simple CSV, and KNN classification only; leave-one-out accuracy is a learning diagnostic, not production validation.
- **Route/files:** `#/lab/ml-model-lab`; `ml-model-lab.js`; `MlModelLab.jsx`.

9. Labs 11–14 and 26 — cloud and networking

Lab 11 — Cloud Regret Pre-Mortem

- **Certificate connection:** Introduction to Cloud Computing.
- **Real user/problem:** a founder or technical lead is considering a cloud migration and needs to expose the ways the decision could become regrettable before committing.
- **Why built:** to turn vague cloud enthusiasm or fear into comparable assumptions, failure scenarios, early warnings, and a reversible pilot.
- **Inputs:** decision context, current/planned cost assumptions, a bounded 12–36 month horizon, and five explicit risk ratings.
- **Decision logic:** compares stay/migrate planning totals from supplied assumptions, constructs exactly five regret scenarios, derives early-warning evidence, and gates a limited pilot with an exit test.
- **Outputs/artifacts:** comparison, five-regret pre-mortem, early warnings, reversible actions, 30-day pilot contract, exit test, and copyable decision memo.
- **Safety/privacy:** browser-local planning; no cloud account, provider API, live quote, pricing promise, migration instruction, or approval.
- **Tests:** `tests/lab11-cloud-regret-premortem.test.mjs` covers bounded inputs, scenario count/content, comparison math, warnings, pilot/exit controls, memo, and invalid input.
- **Limitations:** supplied costs and ratings are assumptions; the tool omits detailed architecture, contracts, taxes, service limits, and organization-specific risk.
- **Route/files:** `#/lab/cloud-regret-premortem`; `cloudRegretPremortem.js`; `CloudRegretPremortem.jsx`.

Lab 12 — AWS Customer-Journey Outage Storyboard

- **Certificate connection:** AWS Cloud Practitioner preparation.
- **Real user/problem:** a small launch team needs a safe GameDay discussion tied to what customers experience, not an unbounded infrastructure failure exercise.
- **Why built:** to connect journey steps and dependency categories to evidence, containment, fallback, recovery proof, and facilitation.
- **Inputs:** customer journey steps, dependency categories, recovery priority, traffic context, and data criticality.
- **Decision logic:** selects three to five journey-relevant failure cards and fills each with symptom, evidence, safe containment, fallback, recovery validation, and a stable AWS concept category.
- **Outputs/artifacts:** outage storyboard/GameDay cards and facilitator checklist.
- **Safety/privacy:** no AWS account, credentials, fault injection, provider call, production command, or promise that the cards cover every failure.
- **Tests:** `tests/lab12-aws-outage-storyboard.test.mjs` covers bounded journey parsing, card selection/count, journey linkage, recovery evidence, checklist output, and validation.
- **Limitations:** dependency categories are planning abstractions; teams must adapt cards to the real architecture and run only authorized non-production exercises.
- **Route/files:** `#/lab/aws-outage-storyboard`; `awsOutageStoryboard.js`; `AwsCustomerJourneyOutageStoryboard.jsx`.

Lab 13 — Azure Access Handoff Simulator

- **Certificate connection:** Microsoft Azure Fundamentals (AZ-900).
- **Real user/problem:** a manager or Azure administrator needs to review joiner, mover, and leaver access handoffs without exposing a tenant or real identities.
- **Why built:** to convert a small roster into owned evidence actions instead of a generic identity checklist.
- **Inputs:** bounded local roster metadata such as status, job/role, resource, privilege, shared-credential indicator, and days since review.
- **Decision logic:** detects orphaned access, role/resource mismatch, excess scope, shared credentials, and review drift, then prioritizes an owner/evidence action.
- **Outputs/artifacts:** issue summary, ordered remediation actions, evidence requirements, and copyable handoff checklist.
- **Safety/privacy:** generic local roster only; no tenant connection, identity, email, credential, permission change, or compliance claim.
- **Tests:** `tests/lab13-azure-access-handoff.test.mjs` covers joiner/mover/leaver findings, shared/excess access, stale review, action ordering, validation, and checklist content.
- **Limitations:** cannot inspect effective permissions, group inheritance, conditional access, logs, or actual completion.
- **Route/files:** `#/lab/azure-access-handoff`; `azureAccessHandoff.js`; `AzureAccessHandoffSimulator.jsx`.

Lab 14 — GCP Transformation Promise Ledger

- **Certificate connection:** Google Cloud Digital Leader preparation.
- **Real user/problem:** an executive sponsor or small-organization project lead needs to challenge a cloud-transformation promise before making a commitment.
- **Why built:** to make an executive claim falsifiable and evidence-gated, rather than producing a generic migration plan or service list.
- **Inputs:** promise statement, target metric, current baseline, claimed target, workload count, data sensitivity, downtime tolerance, dependency complexity, team skill, and modernization intent.
- **Decision logic:** translates the promise into a falsifiable hypothesis, records assumptions/risks, designs the smallest reversible pilot, and creates 30/60/90 evidence gates with stop/continue triggers.
- **Outputs/artifacts:** hypothesis, assumption ledger, evidence needed, pilot, evidence gates, risk owners, inventory checklist, triggers, and copyable sponsor memo.
- **Safety/privacy:** local planning; no GCP account, service deployment, pricing/current-capability claim, guaranteed calendar, or approval. Official GCP sources are used only for stable concepts/names.
- **Tests:** `tests/lab14-gcp-promise-ledger.test.mjs` covers validation, hypothesis construction, assumptions, reversible pilot, evidence gates, triggers, risks, memo, and source boundaries.
- **Limitations:** user-supplied baselines/targets are not verified; evidence gates are review points, not guaranteed delivery dates or transformation proof.
- **Route/files:** `#/lab/gcp-promise-ledger`; `gcp-promise-ledger.js`; `GcpTransformationPromiseLedger.jsx`.

Lab 26 — Network Change Rollback Composer

- **Certificate connection:** The Bits and Bytes of Computer Networking.
- **Real user/problem:** a small-business network administrator planning an IPv4 change needs obvious unsafe address plans blocked and a rollback-ready communication package before touching devices.

- **Why built:** to combine CIDR reasoning with change control; it is intentionally more than a subnet calculator.
- **Inputs:** change goal, downtime window, before inventory, and proposed inventory, with up to 20 named IPv4 network rows.
- **Decision logic:** normalizes /1–/30 CIDRs, calculates usable capacity, detects overlap/reserved-range/capacity blockers, diffs inventories, and withholds execution steps when blocked.
- **Outputs/artifacts:** change findings, precheck, execution or safety interlock, verification, rollback, and stakeholder summary.
- **Safety/privacy:** platform-neutral and local; no device connection, live route/ACL inspection, credentials, vendor commands, or automatic change.
- **Tests:** `tests/lab26-network-change-rollback.test.mjs` covers normalization, a useful example, overlap/capacity blockers, reserved ranges, row limits, and withholding execution on failure.
- **Limitations:** IPv4 only; no IPv6, routing protocol, ACL, DHCP-server, topology, or vendor-specific validation. A network owner must review and rehearse the runbook.
- **Route/files:** `#/lab/network-change-rollback`; `networkChangeRollback.js`; `NetworkChangeRollbackComposer.jsx`.

10. Labs 15–20 — front-end, programming, and data

Lab 15 — Responsive Constraint Handoff

- **Certificate connection:** Introduction to Front-End Development.
- **Real user/problem:** a designer and front-end developer need to agree how dense components reflow across viewports before implementation.
- **Why built:** to turn component and content constraints into a buildable handoff rather than another visual breakpoint toy.
- **Inputs:** screen task, viewport widths, component priority, minimum width, content type, visibility rule, and related constraints.
- **Decision logic:** checks per-viewport feasibility/collisions, chooses stack/reflow/hide behavior, preserves “never hide” items, and derives keyboard order and CSS Grid guidance.
- **Outputs/artifacts:** viewport decision table, visibility/reflow rules, keyboard sequence, CSS starter, and QA checklist.
- **Safety/privacy:** local design metadata; no user tracking, content upload, deployed DOM inspection, or accessibility certification.
- **Tests:** `tests/lab15-responsive-constraint-handoff.test.mjs` covers parsing, feasibility, reflow, required visibility, focus order, generated CSS, QA, and validation.
- **Limitations:** estimates do not replace testing real fonts, content, localization, touch targets, screen readers, zoom, or browser layout.
- **Route/files:** `#/lab/responsive-constraint-handoff`; `responsiveConstraintHandoff.js`; `ResponsiveConstraintHandoff.jsx`.

Lab 16 — Exception-First Python Automator

- **Certificate connection:** Hello Python.
- **Real user/problem:** an office worker or junior developer needs a reviewable batch-file automation starter that anticipates failure and recovery before changing files.
- **Why built:** to teach safe automation through dry-run, path bounds, duplicates, audit, backup, and undo rather than a happy-path script generator.

- **Inputs:** task type, bounded path/glob metadata, naming template, copy/move choice, duplicate strategy, and failure behavior.
- **Decision logic:** validates path and traversal rules, then generates a Python standard-library scaffold with dry run, capped work, checksum/audit behavior, backups, error handling, and undo record.
- **Outputs/artifacts:** copyable Python source and reviewer checklist.
- **Safety/privacy:** the browser does not read/write files or execute Python; traversal such as `..` is rejected and source is withheld on unsafe input.
- **Tests:** `tests/lab16-exception-first-python-automator.test.mjs` covers valid generation, dry-run/audit/undo features, duplicate modes, path rejection, bounds, and invalid input.
- **Limitations:** generated code is a reviewed starter, not a tested production job; filesystem permissions, races, links, encodings, and organization policy need real validation.
- **Route/files:** `#/lab/exception-first-python-automator; exceptionFirstPythonAutomator.js; ExceptionFirstPythonAutomator.jsx`.

Lab 17 — Queue Fairness Replay

- **Certificate connection:** Data Structures in C.
- **Real user/problem:** a helpdesk, clinic, or event manager and a C student need to compare fast urgent service with fairness to earlier arrivals.
- **Why built:** to make queue policy consequences visible and produce a concrete implementation contract.
- **Inputs:** bounded cases with arrival, urgency, and service duration plus an aging setting.
- **Decision logic:** simulates one non-preemptive server under FIFO and priority-with-aging, then calculates dequeue order, wait time, maximum wait, starvation signal, and fairness evidence.
- **Outputs/artifacts:** side-by-side replay, waits/fairness comparison, and copyable C data-structure/algorithm contract.
- **Safety/privacy:** generic cases only; no identities, live queue integration, dispatch action, or claim that the policy is ethically optimal.
- **Tests:** `tests/lab17-queue-fairness-replay.test.mjs` covers parsing, FIFO, aging priority, wait calculations, tie/order rules, starvation/fairness output, and C contract.
- **Limitations:** simplified single server, no preemption, skills, breaks, balking, abandonment, parallel workers, or service-level policy.
- **Route/files:** `#/lab/queue-fairness-replay; queueFairnessReplay.js; QueueFairnessReplay.jsx`.

Lab 18 — Safe C Input Harness

- **Certificate connection:** Programming in C.
- **Real user/problem:** a C student or embedded junior needs bounded console-input code and failure tests before trusting user input.
- **Why built:** to make input validation and memory bounds the primary artifact, not an afterthought to a small program.
- **Inputs:** field definitions with supported type, numeric bounds, maximum text length, and required/optional choice.
- **Decision logic:** validates the field contract, generates `fgets` plus `strtol`/`strtod` parsing patterns, creates boundary/invalid vectors, and estimates declared stack buffers.
- **Outputs/artifacts:** C input scaffold, negative test vectors, stack expectation, and reviewer checklist.
- **Safety/privacy:** generates text only; does not compile, execute, read devices/files, or accept production data.
- **Tests:** `tests/lab18-safe-c-input-harness.test.mjs` covers field parsing, integer/decimal/text generation, bounds, invalid vectors, stack estimate, and malformed definitions.

- **Limitations:** generated code needs compiler/platform review and testing for locale, encoding, integer width, I/O environment, fuzzing, and surrounding program logic.
- **Route/files:** `#/lab/safe-c-input-harness`; `safeCInputHarness.js`; `SafeCInputHarness.jsx`.

Lab 19 — CSV Claim Stress Tester

- **Certificate connection:** Data Analysis with Python.
- **Real user/problem:** a small-business analyst or student has a claimed insight from a CSV and needs to know whether it survives basic data-quality and sensitivity checks.
- **Why built:** profiling/chart tools already exist; this tool tests one explicit business claim and creates a reproducible verification recipe.
- **Inputs:** bounded pasted CSV, numeric outcome column, group column, optional time column, and plain-language claim.
- **Decision logic:** robustly parses rows, applies missingness and sample-size gates, compares group means/medians/effect, repeats after IQR-based outlier trimming, and checks time-slice/subgroup consistency.
- **Outputs/artifacts:** Support, Fragile, or Reject verdict; quality/small-sample/missingness warnings; baseline and sensitivity evidence; and copyable pandas verification recipe.
- **Safety/privacy:** local only; no upload or API. The generated recipe does not embed the supplied rows.
- **Tests:** `tests/lab19-csv-claim-stress-tester.test.mjs` covers parsing, quality failures, supported/fragile/rejected claims, outlier sensitivity, time consistency, recipe generation, and bounds.
- **Limitations:** descriptive deterministic checks do not establish causality, statistical significance, representativeness, or a business decision.
- **Route/files:** `#/lab/csv-claim-stress-tester`; `csv-claim-stress-tester.js`; `CsvClaimStressTester.jsx`.

Lab 20 — LLM Data Contract Firewall

- **Certificate connection:** Generative AI / LLM Architecture and Data Preparation.
- **Real user/problem:** a data engineer needs to decide which declared fields can enter an LLM dataset before any records are processed.
- **Why built:** to create a fail-closed field-level contract for purpose, sensitivity, retention, and leakage tests rather than a generic data-preparation checklist.
- **Inputs:** metadata-only field rows describing name, intended task/purpose, classification, allowed use, and retention.
- **Decision logic:** maps every field to allow, redact, quarantine, or exclude, builds least-data processing stages, adds leakage gates, and withholds permissive output when the contract is incomplete.
- **Outputs/artifacts:** field decision ledger, least-data pipeline, leakage-test contract, strict JSON schema/JSONL record contract, and dataset-card text.
- **Safety/privacy:** accepts field metadata, not records; no LLM, file, API, secrets, production data, or compliance claim.
- **Tests:** `tests/lab20-llm-data-contract-firewall.test.mjs` covers classification/purpose decisions, fail-closed behavior, redaction/quarantine/exclusion, schema and dataset-card generation, leakage checks, and validation.
- **Limitations:** a metadata contract cannot verify actual records, consent, de-identification, retention enforcement, model memorization, legal basis, or dataset quality.
- **Route/files:** `#/lab/llm-data-contract-firewall`; `llmDataContractFirewall.js`; `LlmDataContractFirewall.jsx`.

11. Labs 21–24 — business, finance, and defensive security

Lab 21 — Shortage Response Trade-off Lab

- **Certificate connection:** Microeconomics Principles.
- **Real user/problem:** a small cooperative or shop has scarce essential inventory and must compare a temporary candidate-price policy with a baseline-price purchase cap without receiving a false “optimal” or legal answer.
- **Why built:** to keep access, unmet demand, consumer spend, and segment distribution beside revenue and margin.
- **Inputs:** essential-item description; up to 12 aggregate rows segment | buyers | max units per buyer | willingness to pay; available units; baseline price; unit cost; one candidate price; and a whole-number cap.
- **Decision logic:** candidate-price allocation admits qualifying willingness-to-pay segments and serves higher values first; capped allocation keeps the baseline price, limits buyer demand, and distributes scarcity proportionally. Both use the same inventory and report comparable measures.
- **Outputs/artifacts:** allocated units, revenue, gross-margin proxy, served buyers, unmet baseline demand, average spend, access coverage, segment service-rate gap, assumptions, and copyable owner memo.
- **Safety/privacy:** aggregate rows only; no person-level data or live prices. Willingness to pay is not need/fairness. Output is not pricing/legal advice and requires local essential-goods, anti-gouging, consumer-protection, and rationing review.
- **Tests:** tests/lab21-shortage-response-tradeoff.test.mjs covers both allocation paths, inventory/revenue/spend/access evidence, exclusions, zero access, bounds, unit cost, and memo limitations.
- **Limitations:** one-period static proxy; omits supply response, repeat visits, household need, taxes, enforcement, strategic behavior, and many fairness factors.
- **Route/files:** #/lab/shortage-response-tradeoff; shortageResponseTradeoff.js; ShortageResponseTradeoffLab.jsx.

Lab 22 — LP Exit Rehearsal

- **Certificate connection:** Decentralized Finance (DeFi): The Future of Finance.
- **Real user/problem:** an educational DAO or club-treasury reviewer needs to rehearse an adverse constant-product liquidity-pool exit before a governance decision, without connecting a wallet.
- **Why built:** to put the hold benchmark, break-even fees, price impact, stop rules, and decision evidence around the AMM math.
- **Inputs:** hypothetical initial reserves, proposed deposit, fee assumption, adverse external-price multiplier, and percentage of the owned LP position to withdraw.
- **Decision logic:** uses the proportional deposit and reports unused tokens; derives LP ownership; holds $x \times y = k$ while rebalancing to the scenario price; values the withdrawal in token Y; compares it with holding; and models a small sample swap.
- **Outputs/artifacts:** owned/withdrawn balances, hold comparison, impermanent-loss value/percentage, fee break-even, curve price impact, separate fee-inclusive execution gap, stop triggers, and copyable pre-commitment memo.
- **Safety/privacy:** no wallet, token lookup, live price, approval, transaction, or returns promise; educational scenario only, not investment advice.
- **Tests:** tests/lab22-lp-exit-rehearsal.test.mjs covers adverse-price math, ownership, withdrawal, unused deposit, IL, break-even fees, separate impact/gap measures, stops, memo, source boundaries, and invalid numbers.
- **Limitations:** simplified full-range proportional liquidity; excludes accumulated fees, gas, tax, concentrated ranges, unusual tokens, hooks, vulnerabilities, liquidity changes, and execution conditions.

- **Route/files:** `#/lab/lp-exit-rehearsal`; `lpExitRehearsal.js`; `LpExitRehearsal.jsx`.

Lab 23 — Account Recovery Drill Composer

- **Certificate connection:** Cybersecurity Essentials.
- **Real user/problem:** a family or small-team security coordinator needs to rehearse device, phone, or primary-email loss without exposing credentials or initiating recovery.
- **Why built:** to detect correlated recovery dependencies and produce a secret-free tabletop package before a lockout.
- **Inputs:** up to 20 category-only rows `account label|importance|primary MFA|recovery methods|notification channel`, plus device-loss, phone-loss, or email-loss scenarios.
- **Decision logic:** controlled categories model which recovery/notification methods fail in each scenario, detect single points/shared dependencies, and order verification and owned preparation actions.
- **Outputs/artifacts:** findings, no-secret tabletop steps, verification/notification checklist, generic emergency cards, and owner/evidence actions.
- **Safety/privacy:** rejects email-like labels, URLs, =, long digit sequences, usernames, passwords, recovery codes, secret answers, keys, and real addresses. It never starts recovery; users stop before submitting any provider action.
- **Tests:** `tests/lab23-account-recovery-drill.test.mjs` covers findings, single points, notification gaps, emergency cards, no-secret output, categories, missing scenarios, official sources, and unsafe label rejection including `username=deepan`.
- **Limitations:** cannot verify provider support, backup usability, notification delivery, process changes, or compromise. Owners must verify official provider paths safely.
- **Route/files:** `#/lab/account-recovery-drill`; `accountRecoveryDrill.js`; `AccountRecoveryDrillComposer.jsx`.

Lab 24 — Feature Misuse Contract

- **Certificate connection:** OWASP Top 10 — 2021.
- **Real user/problem:** a startup founder or developer is about to ship one web feature and needs feature-specific negative acceptance tests tied to its customer promise before merge.
- **Why built:** to convert roles and trust-boundary actions into tracked defensive tests, not an OWASP quiz, scanner, attack generator, or generic checklist.
- **Inputs:** feature/customer promise; 2–8 actor roles; selected data categories; and up to eight bounded actor/client/service/storage boundary actions.
- **Decision logic:** deterministically selects and prioritizes five to seven misuse stories. Each connects a broken promise to a safe scenario, relevant OWASP 2021 category, negative acceptance test, observable pass condition, control owner, and evidence.
- **Outputs/artifacts:** prioritized defensive misuse stories and copyable PR acceptance contract/checklist with boundary scope, assumptions, and merge evidence.
- **Safety/privacy:** local metadata only; no target URLs, credentials, secrets, payloads, exploit steps, CVSS, scanning, probing, or contacting real third parties.
- **Tests:** `tests/lab24-feature-misuse-contract.test.mjs` records 8 focused passes covering bounds, deterministic prioritization, complete contracts, sensitive data, safe remote-resource handling, PR output, limits, and official OWASP domains. Full Batch 6 suite: 126 passed.
- **Limitations:** planning artifacts cannot prove coverage, absence of vulnerabilities, control effectiveness, architecture correctness, or compliance; use synthetic test data and qualified review.
- **Route/files:** `#/lab/feature-misuse-contract`; `feature-misuse-contract.js`; `FeatureMisuseContract.jsx`.

12. Labs 25 and 27–30 — advanced defensive workflows

These five labs are functionally implemented and documented, and the cumulative automated suite reached 151 passing tests. At this manual snapshot, their final shared production build plus realistic desktop/mobile/console acceptance remains pending, so the individual notes below distinguish focused implementation evidence from final integrated acceptance.

Lab 25 — Share-Link Afterlife Rehearsal

- **Certificate connection:** IT Security: Defense against the digital dark arts.
- **Real user/problem:** a small-team owner is about to share a sensitive document and needs to know what expiry/revocation can and cannot recover.
- **Why built:** to model the document's afterlife—downloaded copies, screenshots, caches, copied text, and forwarded metadata—rather than only provider permissions.
- **Inputs:** generic artifact label/classification; up to 12 role|business need|view/edit|required days rows; provider mode; permission; expiry; and yes/no residue capabilities.
- **Decision logic:** derives minimum specific-recipient access, required permission, and shortest supported expiry; calculates corrections and a proceed-after-corrections, safer-channel, or do-not-share gate; treats screenshots as non-revocable risk regardless of expectation.
- **Outputs/artifacts:** revocable/non-revocable residue ledger, owner/evidence rows, expiry and former-recipient verification checks, and copyable pre-share/expiry contract.
- **Safety/privacy:** rejects URLs, email-like identities, token-like links, and identifying rows; accepts no file/content/link/token and performs no provider action. Acknowledgement is not proof of deletion.
- **Tests:** tests/lab25-share-link-afterlife.test.mjs records 4/4 focused passes covering access/expiry narrowing, residue, restricted-material blocking, permission downgrade, and invalid metadata. JSX transformation passed.
- **Limitations:** cannot inspect effective permissions, inheritance, guests, sync, logs, retention, legal holds, caches, screenshots, or recipient behavior; not confidentiality/compliance proof.
- **Route/files:** #/lab/share-link-afterlife; shareLinkAfterlife.js; ShareLinkAfterlifeRehearsal.jsx.

Lab 27 — Proof-to-Interview Compiler

- **Certificate connection:** Put It to Work: Prepare for Cybersecurity Jobs.
- **Real user/problem:** a cybersecurity student preparing for an internship has projects and tests but needs every resume/interview statement traceable to evidence.
- **Why built:** to withhold unsupported achievements instead of polishing them into plausible-sounding claims.
- **Inputs:** generic target role; up to eight target tasks; project situation/action/outcome; up to 16 artifact-id|type|what it proves rows; and up to 12 candidate skills.
- **Decision logic:** deterministic term matching links claims/tasks to artifact descriptions. Claims become Backed, Needs evidence, or Do not claim; unsupported numbers and absolute proficiency language are withheld. STAR action/result citations are calculated independently.
- **Outputs/artifacts:** claim ledger, NICE-aligned task map, citation-bearing resume bullets only for backed claims, STAR answer, interviewer evidence packet, and gap drill.
- **Safety/privacy:** rejects identities, emails, URLs, employer/repository links, and unsupported artifact types. It never invents metrics, employers, responsibilities, outcomes, certifications, or hiring promises.

- **Tests:** `tests/lab27-proof-to-interview-compiler.test.mjs` records 5/5 focused passes covering supported citations, unsupported numeric claims, task/STAR/package/gap output, no-bullet rules, and unsafe metadata. JSX transformation passed.
- **Limitations:** term overlap is not semantic verification; artifact descriptions are user assertions and the tool cannot inspect, run, authenticate, or grade them or predict hiring.
- **Route/files:** `#/lab/proof-to-interview-compiler`; `proofToInterviewCompiler.js`; `ProofToInterviewCompiler.jsx`.

Lab 28 — Containment Side-Effect Ledger

- **Certificate connection:** Introduction to Cybersecurity Tools & Cyberattacks.
- **Real user/problem:** a junior SOC analyst has a plausible alert and candidate containment actions but lacks authority to trade customer availability for security without evidence and owner review.
- **Why built:** to put evidence confidence, customer-promise harm, reversibility, rollback proof, and authorization in one pre-action ledger.
- **Inputs:** generic alert claim; affected service/customer promise; up to 20 `fact|source|category|confidence` rows; and up to 12 `action|reversibility|service|harm|owner` rows.
- **Decision logic:** requires at least one high-confidence fact and two source categories before high-harm action becomes reviewable; otherwise blocks/escalates. Irreversible actions are always “escalate only.”
- **Outputs/artifacts:** per-action disposition, customer side effect, evidence to collect, authorization gate, recovery proof, escalation trigger, owner, evidence gaps, and copyable analyst handoff.
- **Safety/privacy:** rejects URLs, address/hash-like indicators, credentials, payload/script/query/command content, and contact-address owners. It connects to no telemetry/system and executes or recommends no real containment action.
- **Tests:** `tests/lab28-containment-side-effect-ledger.test.mjs` records 4/4 focused passes for complete ledgers, never recommending irreversible action, blocking weak-evidence high harm, and unsafe-input rejection. JSX parse and diff check passed.
- **Limitations:** does not establish incident truth, evidence authenticity, action effectiveness, acceptable customer harm, or authorization; confidence/diversity thresholds are educational baselines.
- **Route/files:** `#/lab/containment-side-effect-ledger`; `containmentSideEffectLedger.js`; `ContainmentSideEffectLedger.jsx`.

Lab 29 — Detection Contract Drift Guard

- **Certificate connection:** Automate Cybersecurity Tasks with Python.
- **Real user/problem:** a SOC engineer needs to detect when a producer schema change silently changes the meaning of a Python-based detection.
- **Why built:** to compare one detection claim's field/type/unit/time assumptions and create a regression contract without ingesting logs or connecting to a SIEM.
- **Inputs:** detection claim; up to 50 `field|required|yes/no|type|unit` assumptions; old/new schema rows `field|type|unit`; and a trailing `?` convention for newly optional new-schema fields.
- **Decision logic:** classifies required disappearance/type incompatibility as invalid drift; optional disappearance/unit change as silent drift; newly optional/additional/compatible-but-changed fields as widened drift. Each impact receives repair evidence and bounded vectors.
- **Outputs/artifacts:** drift ledger, affected claim, repair contract, valid/missing/wrong-type vectors, and copyable standard-library `unittest` Python harness with a 64-field guard.
- **Safety/privacy:** metadata only; rejects queries, executable content, URLs, credentials, secrets, records, and unsupported identifiers. It does not run Python or connect to a source.

- **Tests:** `tests/lab29-detection-contract-drift-guard.test.mjs` records 4/4 focused passes for classifications, repair/vectors, bounded standard-library source, and validation. Generated Python passed in-memory `compile()` syntax only; it was not executed.
- **Limitations:** cannot prove event completeness, values, time correctness, semantics, false-positive/negative behavior, or detection coverage; ? is a lab convention.
- **Route/files:** `#!/lab/detection-contract-drift-guard; detectionContractDriftGuard.js; DetectionContractDriftGuard.jsx.`

Lab 30 — Assurance Change Shockwave Mapper

- **Certificate connection:** Play It Safe: Manage Security Risks.
- **Real user/problem:** a small-company security lead has a vendor, system, configuration, or owner change and needs to know which evidence-backed control claims and accepted-risk decisions require review.
- **Why built:** to start from one operational change and trace its evidence shockwave, not create another static risk register or audit dashboard.
- **Inputs:** change category/description; up to 20 `claim-id|promise|owner|review cadence days` rows; up to 40 `evidence-id|claim-id|type|age days|dependency categories` rows; and up to 20 `risk-id|claim-id|days until expiry` rows.
- **Decision logic:** matching change dependency invalidates that evidence chain; age at/over cadence needs review; otherwise evidence is current from supplied metadata. Claims inherit the worst linked evidence state. Risk decisions are expired at ≤ 0 days, due soon at 1–30, active above 30. Renewal order uses visible state/expiry/overdue/ID tie rules and no opaque score.
- **Outputs/artifacts:** claim dispositions, evidence shockwave, renewal instructions/order, risk-decision windows, owner/evidence needs, assumptions, and copyable executive memo.
- **Safety/privacy:** generic metadata only; rejects unknown links, URLs, and credential-like values; no system/evidence inspection or integration. Never certifies compliance, approves an audit/change, accepts risk, or renews a decision.
- **Tests:** `tests/lab30-assurance-change-shockwave.test.mjs` records 8/8 focused passes for parsing, safety validation, dependency propagation, cadence boundary, expiry labels, deterministic no-score order, safe memo language, and NIST source domains. JSX parse and diff check passed.
- **Limitations:** only as complete as supplied metadata; cannot prove control failure/effectiveness, evidence authenticity/sufficiency, complete dependencies, or appropriate risk acceptance.
- **Route/files:** `#!/lab/assurance-change-shockwave; assuranceChangeShockwave.js; AssuranceChangeShockwaveMapper.jsx.`

13. Verification and acceptance

Automated gate

For every code change:

- 1 run the focused test file;
- 2 validate malformed, boundary, and deterministic paths, not only the sample;
- 3 run `pnpm run test` to validate the manifest and full Node suite;
- 4 run the relevant production build;
- 5 before a batch closes, run `pnpm run build` for all public surfaces.

The most recent recorded cumulative test result is **151 passing tests** for Batch 7 source integration. This is a checkpoint count, not a permanent promise: adding or changing tests will change it. At the same snapshot, the last

fully recorded all-surface production build/browser acceptance belongs to Batch 6 at 126 tests. Batch 7 still needs that central gate.

Browser gate

For a realistic example on every changed route:

- load the direct hash route, not only catalog navigation;
- use example/reset and the primary calculate/generate action;
- trigger one meaningful validation or safety path;
- verify copy/download/print behavior when the lab promises it;
- reload when persistence is part of the product;
- inspect the console for warnings/errors;
- check keyboard focus, labels, instructions, and result announcements;
- inspect at a 390 × 844 viewport and a desktop viewport;
- confirm dense tables are locally scrollable and do not create page-level overflow.

The completed batch reports record successful browser and console checks through Labs 01–24 and 26. Do not copy that claim to Labs 25 and 27–30 until their pending gate is actually run and documented.

Functional acceptance questions

A lab does not pass merely because it renders. Ask:

- Can the intended user name the decision or artifact they get?
- Does the example produce something they can reuse outside the page?
- Are calculations and tie-breaks inspectable?
- Does invalid input withhold misleading output?
- Are privacy and authority boundaries visible before action?
- Does the output avoid claims the model cannot establish?
- Can a reviewer reproduce important results from tests?

CI

`.github/workflows/frontend-tests.yml` uses Node 22, pnpm 11.19.0, a frozen lockfile install, and pnpm run build on pushes and pull requests to main/master. This is the repository's clearest currently recorded runtime reference even though `package.json` itself has no `engines` field.

The separate `.github/workflows/python-tests.yml` runs Python 3.12 and legacy Python unit discovery. That workflow belongs to older repository utilities and should not be confused with the Node Applied Labs suite.

14. Deployment and rollback

Public surfaces

The intended production topology is:

Surface	Build/source boundary	Intended hosting state at snapshot
Portfolio	portfolio/ from portfolio-src/	existing GitHub Pages portfolio is linked by the user, but redesigned source deployment is not recorded complete

Surface	Build/source boundary	Intended hosting state at snapshot
Applied Labs	labs/ from labs-src/	one static deployment intended; final production URL not recorded
Timber demo	timber-demo/ from timber-src/	static deployment intended; exact rebuilt-demo URL not verified in project control
Printing ERP demo	separate private repo/build	Cloudflare Workers Static Assets URL recorded live

Because Vercel repository authorization failed for the ERP, that application was moved to Cloudflare Static Assets. This does not automatically prove that every other surface has been deployed to Cloudflare. Record platform, project, commit, version/deployment ID, URL, and smoke-test result for each surface separately.

Deployment sequence

- 1 Ensure the working tree contains no client database, real export, credentials, or unreviewed generated artifact.
- 2 Run `pnpm install --frozen-lockfile` in CI or the documented equivalent.
- 3 Run `pnpm run build`.
- 4 Inspect `portfolio/`, `labs/`, and `timber-demo/` for expected entry files and assets.
- 5 Deploy each output to its explicitly configured static project.
- 6 Verify the portfolio home, a direct Applied Labs hash route, the Timber sample/save/reopen/print path, and ERP /owner plus a deep route.
- 7 Record the commit, deployment/version ID, URL, time, and accepted checks in project control.
- 8 Keep the previous known-good deployment available until smoke tests pass.

No Cloudflare command/configuration for the portfolio, labs, or Timber is recorded in this repository, so this manual intentionally does not invent a CLI command. Use the hosting project's actual configuration when it is added.

Rollback

- Re-promote the last known-good static deployment/version rather than editing built files in production.
- For a bad manifest or route integration, revert the specific source change, rebuild all surfaces, and re-run the direct-route smoke tests.
- For Timber, never “repair” a formula in production without restoring the golden business vectors.
- For ERP, public demo rollback must not enable live mode or reuse live secrets. Cloudflare version 4bd9ebd0-69d8-4c3f-b238-947fde815cc5 is the recorded accepted version at this snapshot.
- Document the failure symptom, affected route/version, rollback result, and follow-up owner.

15. Maintenance guide

Adding or changing a lab

- 1 Start with the certificate concept, real user, problem, and reusable artifact.
- 2 Check adjacent/common tools and write the exact workflow differentiator; do not claim worldwide novelty.
- 3 Define bounded inputs, deterministic rules, safety limits, and validation before UI work.
- 4 Keep pure logic in a lab-specific `.js` module and UI in `.jsx`.
- 5 Add or update a unique focused test in `tests/`.
- 6 Add the lazy route in `labs-src/main.jsx` and the canonical entry in `project-manifest.json` only through the shared integration workflow.
- 7 Add/update the project-control lab document with source boundaries and limitations.

- 8 Run focused tests, full suite, build, realistic browser flow, mobile, keyboard, and console checks.
- 9 Update status/report only after evidence exists.

Dependency maintenance

- Keep `pnpm-lock.yaml` reviewed and committed with dependency changes.
- Run the full combined build after React, Vite, animation, or plugin changes.
- Treat security audit output as a triage input, not permission for an untested bulk upgrade.
- The separate ERP's 2 moderate and 5 high advisories require a dedicated repair/test pass.
- Avoid adding APIs or large runtimes to labs whose value depends on reliable no-key local execution.

Data hygiene

- Never commit `.db`, `.db-wal`, `.db-shm`, `profile_data.json`, secrets, or imported client ZIP content.
- Keep examples fictional and visibly labelled.
- Do not paste real credentials, customer identities, logs, telemetry, wallet data, recovery codes, account addresses, or production evidence into labs.
- Inspect exported/copied artifacts so they contain only intended generic metadata.
- Browser-local storage is not a secure vault, backup, or cross-device database.

Documentation maintenance

Update this manual only from evidence. If a new deployment is made, add its URL, platform, commit, version ID, and smoke-test result. If the code changes a formula or decision threshold, update the lab section and tests together. If a detail cannot be verified, write **unknown** or **pending** rather than copying an assumption from a screenshot or old README.

16. Troubleshooting

A lab route shows the catalog or not-found state

- Confirm the route is exactly `#/lab/<manifest-slug>`.
- Confirm the slug matches `docs/project-control/project-manifest.json` and the registry in `labs-src/main.jsx`.
- Confirm the lazy import filename and exported React component match.
- Run the Applied Labs build to catch missing modules or JSX errors.
- For normal path-based SPA deep links on other applications, confirm the host's fallback; hash lab routes should not require a server rewrite.

Manifest validation fails

- Run `node scripts/validate_project_manifest.mjs` directly.
- Check duplicate IDs/slugs, total count, status values, certificate mapping, and flagship count.
- Do not “fix” the validator to accept an accidental manifest shape; reconcile source and intended schema.

A test passes alone but full build fails

- Run `pnpm run test`, then the specific `build:*` command to isolate whether the problem is manifest, cross-test state, JSX, import path, or bundling.

- Check filename case; Linux CI is case-sensitive even when the Windows workspace is forgiving.
- Check for shared-file conflicts from parallel work.
- Use the Vite error's first source location, not only its final stack frame.

The page overflows on mobile

- Inspect the page's `scrollWidth` versus `clientWidth` at 390 px.
- Check grid/flex children for missing `min-width: 0`.
- Contain wide tables within their own horizontal scroll region.
- Avoid fixed pixel widths in copied code/memo panels.
- Re-run the realistic workflow after the CSS fix; an empty form may not expose result-table overflow.

Copy or download does not work

- Clipboard APIs may require a secure context and browser permission; provide visible success/failure state and manual-select fallback where possible.
- For SVG export, verify XML escaping, metadata, filename, and Blob/object-URL cleanup.
- For print/PDF, use the browser print dialog; do not claim a PDF was created until the user saves it.

Browser-local records disappear or corrupt

- Confirm the same browser/profile/origin is in use.
- Browser storage can be cleared by the user or policy and is not a backup.
- Use the product's reset/recovery behavior; never connect the public demo to a private database to “restore” sample state.
- For Timber, test sample → save → reset → reopen again after storage changes.

Timber totals differ from an ordinary calculator

- Confirm business inches use millimetres ÷ 20.
- Confirm positive dimensions round upward to 0.5 inch with a 1.5-inch minimum.
- Confirm CFT uses rounded billing dimensions while M3 uses raw millimetres and actual metric length.
- Confirm line amounts and invoice totals apply the specified rounding order.
- Run node `--test tests/timber-domain.test.mjs` before changing code.

ERP dashboard hangs or shows no demo data

- Confirm the public deployment is explicitly in demo mode.
- Confirm demo initialization creates no Supabase client/request.
- Inspect corrupt local demo storage recovery.
- Do not add a temporary owner bypass or synthetic fallback to live mode.
- Compare the deployment with the recorded accepted Cloudflare version and private ERP runbook.

Deployment appears successful but a direct route fails

- Test the exact production URL after deployment, including /owner for ERP and a direct lab hash route.
- For path-routed SPAs, configure a static-asset fallback to the app entry.
- Record the deployed commit/version; otherwise it is impossible to distinguish a stale deployment from a source bug.

17. How to explain the project in an interview

Thirty-second version

I converted 30 course certificates into a single Applied Labs product and kept two larger business applications as flagships. I used a functional-first process: each lab has bounded input, transparent local logic, a useful artifact, validation, tests, and honest limitations before any visual redesign. The two flagships demonstrate privacy-aware demo architecture—browser-local synthetic modes for public review and strict separation from private business data.

Two-minute architecture version

Explain four decisions:

- 1 **One catalog, 30 lazy routes:** easier deployment and maintenance than 30 sites, while every lab remains independently testable.
- 2 **Pure logic separated from React:** Node tests can verify formulas, parsing, tie-breaks, and safety gates without browser automation.
- 3 **Local-first demonstrations:** no API key, wallet, cloud account, scanner, or secret is needed for the core result; demos remain reliable and privacy-preserving.
- 4 **Truthful product boundaries:** simulated attention is not a language model, SVG generation is not a trained GAN, planning tools do not certify compliance, and checkpoint test counts are not user-impact metrics.

Flagship story

For Printing ERP, describe the original failure (public dashboard stuck on empty loading skeletons), the architectural fix (explicit demo/live runtime split), the security rule (demo never touches Supabase; live fails closed), persistence verification, and Cloudflare direct-route deployment.

For Timber, explain that the hardest part was preserving a client's nonstandard but intentional business-inch rule while adding a complete billing domain. Show the difference between rounded CFT billing dimensions and raw-millimetre M3, then explain synthetic local storage and why the private SQLite database was excluded.

Lab story pattern

Use this structure for any lab:

- 1 “The certificate concept was ...”
- 2 “The real user was ... and their decision/problem was ...”
- 3 “I avoided a generic quiz/calculator by making the output ...”
- 4 “The deterministic rule is ...”
- 5 “The main safety/privacy boundary is ...”
- 6 “The focused test proves ...”
- 7 “It still cannot prove ...”

Example for Lab 24:

OWASP lists security categories, but a startup developer still needs merge-ready tests for one feature. The lab starts from a customer promise and trust-boundary actions, generates safe misuse stories, maps them to relevant OWASP 2021 categories, and produces observable

negative acceptance tests with owners and evidence. It never scans or generates exploit payloads, and the output does not prove the feature is secure.

Evidence to show

- a live flagship workflow and its privacy statement;
- one Applied Lab's example, validation path, and copied/exported artifact;
- the pure domain module next to its focused Node test;
- the canonical manifest and lazy route;
- a batch report showing what passed and what remained pending;
- a limitation statement that prevents overclaiming.

Never say “30 production AI systems,” “world-first,” “fully secure,” “compliant,” or “deployed everywhere.” Say “30 functional, deterministic applied prototypes,” then distinguish the completed acceptance evidence from pending visual/deployment work.

18. Appendices

Appendix A — canonical lab routes, sources, and tests

All component/model paths below are under `labs-src/labs/`; all tests are under `tests/`.

#	Route slug	Domain module	React component	Focused test
01	ai-workflow-canvas	aiWorkflowCanvas.js	AIWorkflowCanvas.jsx	lab01-ai-workflow-canvas.test.mjs
02	mini-rag-studio	miniRagStudio.js	MiniRagStudio.jsx	lab02-mini-rag-studio.test.mjs
03	gan-latent-gallery	ganLatentModel.js	GanLatentGallery.jsx	lab03-gan-latent.test.mjs
04	cnn-feature-explorer	cnnConvolution.js	CnnFeatureExplorer.jsx	lab04-cnn-convolution.test.mjs
05	attention-text-explorer	attention-text-explorer.js	AttentionTextExplorer.jsx	attention-text-explorer.test.mjs
06	neural-network-playground	neuralNetworkPlayground.js	NeuralNetworkPlayground.jsx	lab06-neural-network-playground.test.mjs
07	genai-lifecycle-explorer	genAiLifecycleExplorer.js	GenAiLifecycleExplorer.jsx	lab07-genai-lifecycle-explorer.test.mjs
08	ai-opportunity-scorer	aiOpportunityScorer.js	AiOpportunityScorer.jsx	lab08-ai-opportunity-scorer.test.mjs
09	prompt-workbench	promptWorkbench.js	PromptWorkbench.jsx	lab09-prompt-workbench.test.mjs
10	ml-model-lab	ml-model-lab.js	MlModelLab.jsx	lab10-ml-model-lab.test.mjs
11	cloud-regret-premortem	cloudRegretPremortem.js	CloudRegretPremortem.jsx	lab11-cloud-regret-premortem.test.mjs
12	aws-outage-storyboard	awsOutageStoryboard.js	AwsCustomerJourneyOutageStoryboard.jsx	lab12-aws-outage-storyboard.test.mjs
13	azure-access-handoff	azureAccessHandoff.js	AzureAccessHandoffSimulator.jsx	lab13-azure-access-handoff.test.mjs
14	gcp-promise-ledger	gcp-promise-ledger.js	GcpTransformationPromiseLedger.jsx	lab14-gcp-promise-ledger.test.mjs
15	responsive-constraint-handoff	responsiveConstraintHandoff.js	ResponsiveConstraintHandoff.jsx	lab15-responsive-constraint-handoff.test.mjs

#	Route slug	Domain module	React component	Focused test
16	exception-first-python-automator	exceptionFirstPythonAutomator.js	ExceptionFirstPythonAutomator.jsx	lab16-exception-first-python-automator.test.mjs
17	queue-fairness-replay	queueFairnessReplay.js	QueueFairnessReplay.jsx	lab17-queue-fairness-replay.test.mjs
18	safe-c-input-harness	safeCInputHarness.js	SafeCInputHarness.jsx	lab18-safe-c-input-harness.test.mjs
19	csv-claim-stress-tester	csv-claim-stress-tester.js	CsvClaimStressTester.jsx	lab19-csv-claim-stress-tester.test.mjs
20	llm-data-contract-firewall	llmDataContractFirewall.js	LlmDataContractFirewall.jsx	lab20-llm-data-contract-firewall.test.mjs
21	shortage-response-tradeoff	shortageResponseTradeoff.js	ShortageResponseTradeoffLab.jsx	lab21-shortage-response-tradeoff.test.mjs
22	lp-exit-rehearsal	lpExitRehearsal.js	LpExitRehearsal.jsx	lab22-lp-exit-rehearsal.test.mjs
23	account-recovery-drill	accountRecoveryDrill.js	AccountRecoveryDrillComposer.jsx	lab23-account-recovery-drill.test.mjs
24	feature-misuse-contract	feature-misuse-contract.js	FeatureMisuseContract.jsx	lab24-feature-misuse-contract.test.mjs
25	share-link-afterlife	shareLinkAfterlife.js	ShareLinkAfterlifeRehearsal.jsx	lab25-share-link-afterlife.test.mjs
26	network-change-rollback	networkChangeRollback.js	NetworkChangeRollbackComposer.jsx	lab26-network-change-rollback.test.mjs
27	proof-to-interview-compiler	proofToInterviewCompiler.js	ProofToInterviewCompiler.jsx	lab27-proof-to-interview-compiler.test.mjs
28	containment-side-effect-ledger	containmentSideEffectLedger.js	ContainmentSideEffectLedger.jsx	lab28-containment-side-effect-ledger.test.mjs
29	detection-contract-drift-guard	detectionContractDriftGuard.js	DetectionContractDriftGuard.jsx	lab29-detection-contract-drift-guard.test.mjs
30	assurance-change-shockwave	assuranceChangeShockwave.js	AssuranceChangeShockwaveMapper.jsx	lab30-assurance-change-shockwave.test.mjs

Appendix B — command reference

```
# Install
pnpm install

# Develop one surface
pnpm run dev:portfolio
pnpm run dev:labs
pnpm run dev:timber

# Validate canonical manifest only
node scripts/validate_project_manifest.mjs

# Full Node/manifest test gate
pnpm run test

# Example focused tests
node --test tests/lab01-ai-workflow-canvas.test.mjs
node --test tests/lab19-csv-claim-stress-tester.test.mjs
node --test tests/lab30-assurance-change-shockwave.test.mjs
node --test tests/timber-domain.test.mjs

# Build one or all public surfaces
pnpm run build:portfolio
pnpm run build:labs
pnpm run build:timber
pnpm run build

# Preview outputs
pnpm run preview
pnpm run preview:labs
pnpm run preview:timber
```

Appendix C — project-control source index

- docs/project-control/STATUS.md — current checkpoint and known risks.
- docs/project-control/DECISIONS.md — locked scope, deployment, privacy, and modeling decisions.
- docs/project-control/EXECUTION_PLAN.md — batch strategy and acceptance sequence.
- docs/project-control/ACCEPTANCE_CRITERIA.md — per-lab, flagship, batch, and final gates.
- docs/project-control/project-manifest.json — canonical 30-lab and 2-flagship inventory.
- docs/project-control/TIMBER_BILLING_SPEC.md — locked measurement and billing rules.
- docs/project-control/PRINTING_ERP_DEMO_SPEC.md — demo/live runtime and acceptance boundary.
- docs/project-control/batches/ — implementation and verification evidence by batch.
- docs/project-control/labs/ — detailed purpose, logic, safety, sources, tests, and limitations for Labs 01–30.

Appendix D — known open work at this snapshot

- 1 Run and record Batch 7's central production build and realistic browser/mobile/console acceptance.
- 2 Run final responsive browser, link, interaction, and console acceptance for the redesigned portfolio and 30-lab catalog.
- 3 Verify and record the production URL/version for the rebuilt Timber demo.
- 4 Deploy and record the Portfolio and Applied Labs production base URLs.
- 5 Refresh generated resume links with the verified production portfolio URL.
- 6 Run final cross-surface regression, accessibility, link, console, and health checks.
- 7 Triage and safely remediate the separate ERP dependency advisories.

- 8 Reconcile the root README with the current portfolio mission.

Completing an item means attaching evidence—test result, build result, deployed version, browser check, or updated source—not simply changing its status label.